

Recursive Large Language Models

Ryan Lee

October 5, 2026

Abstract

I summarize recent work on recursive large language model (LLM) architectures - instead of adding unique transformer layers to sequentially transform the input into generated sequences, these models use a small, fixed set of transformer layers, calling this one set repeatedly until the input representations are sufficiently transformed. By varying the number of self-calls to a singular set of transformer layers, a model can modify its own effective depth based on the difficulty of the input, thinking harder for steps which are more difficult and thinking less when the step is easy. In this review, I'll describe how these recurrent-depth architectures are trained, specifically focusing on their training objective—the mathematical formulation of the optimization problem. These recurrent-depth architectures are shown out-compete much larger language models: on standard benchmarks, ByteDance's Ouro 1.4B parameter model outperforms larger, 3B parameter models such as Qwen2.5 and Llama 3.2 3B. At the end, I'll outline some future directions that I'm interested in looking into.

1 Introduction

The dominant paradigm in large language model development has long relied on scaling model size as the primary path to improved performance, accompanied by proportional increases in data and compute. However, recent work has begun to explore an orthogonal direction: achieving dynamic computation within a fixed parameter budget through architectural innovation. Rather than stacking ever-deeper unique transformer layers, recursive language model architectures reuse a small, fixed set of layers repeatedly, allowing models to adaptively allocate computational depth based on input difficulty. This approach represents a fundamental shift from viewing depth as a static architectural choice to treating it as a dynamic resource that can be tuned at inference time.

Early work in this direction introduced the **Hierarchical Reasoning Model (HRM)**, shown in Fig. 1, a dual-network architecture with two coupled recurrent modules operating at different timescales: a slow high-level module for abstract strategic planning and a fast low-level module for detailed tactical execution, achieving hierarchical convergence that avoids the early saturation issues of standard recurrent networks. As detailed in Fig. 2, the **Tiny Recursive Model (TRM)** subsequently challenged the necessity of such hierarchical complexity, demonstrating that a dramatically simplified approach using just a single tiny two-layer network can achieve superior generalization through straightforward recursive refinement of predicted answers and latent reasoning states.

Building on these insights, the **Looped Language Model**, shown in Fig. 3, scales this recursive approach to practical language model pretraining, building reasoning capability directly into the pre-training phase through iterative computation in latent space and demonstrating that small parameter-efficient models can match much larger standard transformers across diverse benchmarks. In this project, I describe how recurrent-depth architectures are trained, specifically focusing on their training objective—the mathematical formulation of the optimization problem that enables these models to learn how deeply to reason.

2 Recurrent-Depth Architectures

2.1 Hierarchical Reasoning Models

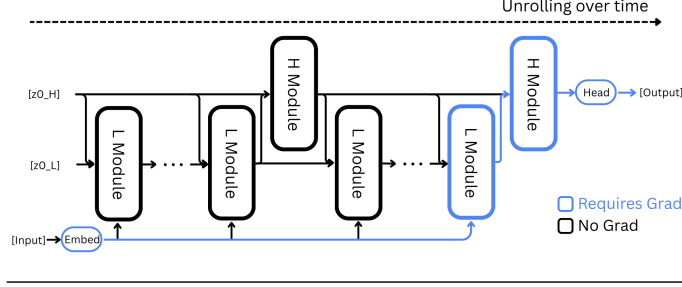


Figure 1: In a HRM, a low-level (L) modules and a High-level (H) module are configured in series. The module sequence above is repeated N_{sup} times before decoding output to answer space.

The Hierarchical Reasoning Model (HRM) [WLS⁺25] employs a dual-network architecture with two coupled recurrent modules operating at different timescales (Figure 1): a high-level (H) module for abstract planning and a low-level (L) module for detailed computation. The L module receives input ($embed + z_L + z_H$) and iterates for a fixed number of steps, then the H module updates once. In practice, the authors employ this process, called **deep supervision**, by repeating the 2L-1H sequence N_{sup} times per example, reusing previous z_L and z_H states. HRM uses **one-step gradient approximation** to avoid expensive backpropagation through time. Assuming fixed-point convergence where $z_L^* \approx f_L(z_L^* + z_H + x)$ and $z_H^* \approx f_H(z_L + z_H^*)$, gradients are computed only from final states. As shown in Figure 1, intermediate modules are detached (black boxes) while only final modules and the prediction head require gradients (blue boxes), reducing memory from $O(T)$ to $O(1)$. HRM also incorporates **Adaptive Computation Time (ACT)** for learned early stopping.

2.2 Tiny Recursive Models

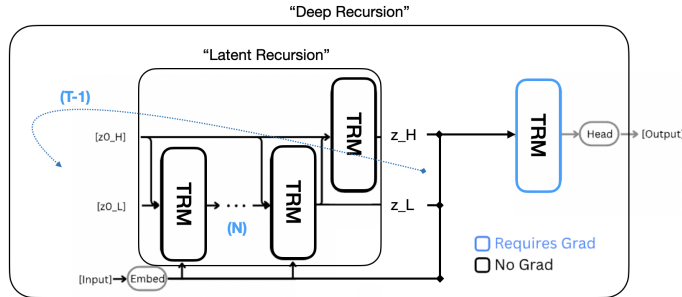


Figure 2: In a TRM, a single stack of transformer layers serves as both the L module and H module.

The Tiny Recursive Model (TRM) [JM25] simplifies the recursive reasoning approach by using a single tiny two-layer network that iteratively refines both a predicted answer y and a latent reasoning state z (Figure 2). Unlike HRM’s dual-network hierarchy, TRM learns **two modes** within one model: a feature prediction mode (z_L) and an embedded solution mode (z_H). The model recursively applies the same network N times in **latent recursion**, updating its internal representations, then passes through a final transformation in the **deep recursion** phase to produce the output. The model employs **deep supervision** by looping the complete process multiple times per example, saving and reusing z_H and z_L from previous iterations as initialization. As shown in Figure 2, only the final TRM module and head require gradients (blue boxes), while intermediate recursions are detached (black boxes). With only 7M parameters, TRM achieves 45% on ARC-AGI-1 and 87% on Sudoku-Extreme, outperforming the 27M parameter HRM and larger chain-of-thought models.

2.3 Looped Language Models

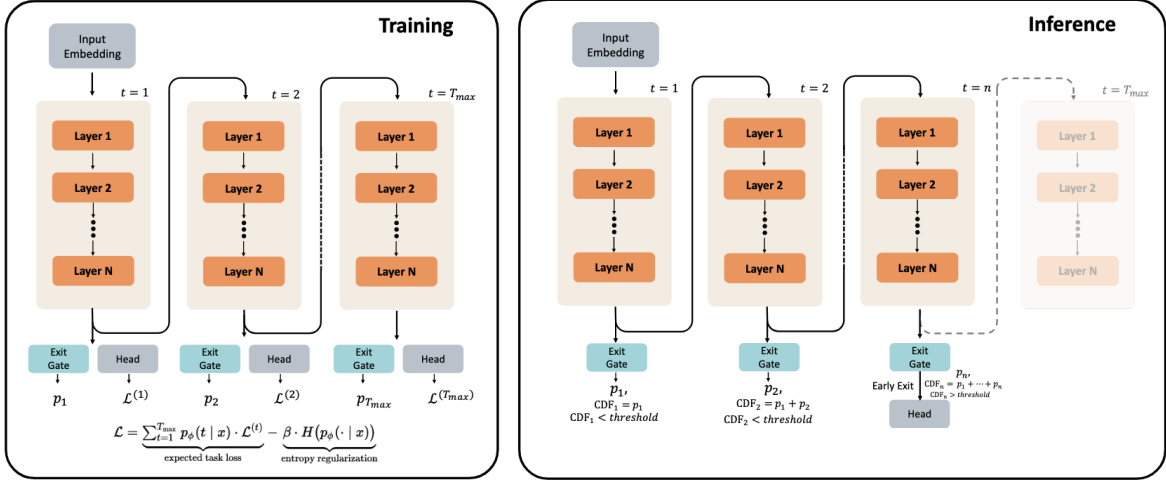


Figure 3: Overview of Ouro, ByteDance’s recurrent-depth Model. Left (Training): During training, the model applies the same stack of N layers for T_{max} recurrent steps. At each recurrent step t , an exit gate predicts the probability p_e of exiting, and a language modeling head L_e computes a prediction loss. Right (Inference): During inference, the model can exit early based on accumulated exit probability.

The Looped Language Model (LoopLM) [ZWH⁺25] also applies a set of transformer layers recursively with adaptive computational depth. Similar to the individual modules in the HRM or TRM architectures, LoopLM uses weight-tied layers where the same parameters are reused across recurrent steps, allowing the model to, during inference, dynamically exit early based on input difficulty. The model processes input through T recurrent iterations: $h^{(t+1)} = f_{\theta}(h^{(t)})$ for $t = 0, \dots, T - 1$, where f_{θ} represents the shared transformer block. The authors frame this process as adaptive reasoning in the latent space, as the latent embeddings are continually transformed by a recursive calls.

LoopLM incorporates two unique components beyond the recurrent transformer block: a **language modeling head** that produces token predictions at each recurrent step, and a **learned gating mechanism** that determines when to halt computation. At each step t , a gate produces a halting probability, and cumulative probability $\text{CDF}(t) = \sum_{i=1}^t p_i$ is computed. The model exits when $\text{CDF}(t) > \tau$ for a deployment-time threshold τ , allowing dynamic depth allocation based on input difficulty. This differs from the Adaptive Computation Time (ACT) used in HRM and TRM, where the model learns to halt by including a “ponder cost” penalty in the training loss that explicitly discourages long computation. LoopLM’s approach instead uses the threshold q as a tunable hyperparameter at inference time: the gate is trained to estimate when additional computation helps, but the actual stopping decision is controlled by q after training is complete.

Unlike TRM’s or HRM’s detached intermediate states, LoopLM employs full backpropagation through all recurrent steps used during training, with gradients flowing through the entire computational graph. As described in more detail in the later section, LoopLM uses a two-stage training approach: Stage I pretrains the model with an entropy-regularized objective using a uniform prior to enable unbiased exploration of all depths, while Stage II focuses exclusively on training the adaptive exit gates to correlate stopping decisions with actual performance improvements. With 1.4B and 2.6B parameters trained on 7.7T tokens, LoopLM matches the performance of 4B and 8B standard transformers respectively. Full results are reported in the Appendix.

3 Problem Formulation

In all recurrent-depth architectures, the training objective contains the mathematical formulation of the critical learning task for recursive models: learning to identify the sufficient input transformation or the “base case” given as input the state of the embedding and the outputs of previous recursive calls. The authors of LoopLM formally define the recursive architecture as following:

Let $\text{emb}(\cdot) : \mathbb{R}^{|V|} \rightarrow \mathbb{R}^d$ be the token embedding; $\mathcal{T}_\theta(\cdot) : \mathbb{R}^{M \times d} \rightarrow \mathbb{R}^{M \times d}$ a causal transformer layer parameterized by θ with hidden size d and input length M , and $\text{lmhead}(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^{|V|}$ be the unembedding layer with vocabulary size V . Let $t \in \{1, \dots, T_{\max}\}$ be the number of loop steps (the number of **recurrent steps** or **recurrent depth**). The looped model $F^{(t)}$ reuses the same depth- L layer stack t times:

$$F^{(t)}(\cdot) = \text{lmhead} \circ \underbrace{\mathcal{M}^L \circ \mathcal{M}^L \circ \dots \circ \mathcal{M}^L}_{t \text{ iterations}} \circ \text{emb}(\cdot). \quad (1)$$

They also define an probability of exiting after a given depth-L layer stack, as learned by the exit gate. Given that each step has a unique exit probability, we can calculate the probability of exiting at step t also as a function of the previous step probabilities:

At each loop t , the gate outputs an exit probability $\lambda_t(x) = \sigma(\text{Linear}_\phi(h^{(t)})) \in (0, 1)$ where $h^{(t)}$ is the final-layer hidden state and ϕ are the gate parameters. The unnormalized probability of exiting first at step t is:

$$\tilde{p}_t(x) = \lambda_t(x) \prod_{j=1}^{t-1} (1 - \lambda_j(x)), \quad t = 1, \dots, T_{\max} - 1, \quad (2)$$

representing the probability of exiting at t after surviving steps 1 through $t - 1$. To obtain a valid distribution, we assign the remaining mass to the final step:

$$p_\theta(t | x) = \begin{cases} \tilde{p}_t(x), & t = 1, \dots, T_{\max} - 1, \\ \prod_{j=1}^{T_{\max}-1} (1 - \lambda_j(x)), & t = T_{\max}, \end{cases} \quad \text{so} \quad \sum_{t=1}^{T_{\max}} p_\theta(t | x) = 1. \quad (3)$$

In LoopLM, the pretraining task is next token prediction, with prediction loss defined as the probability weighted sum of step loss, with a entropy regularization term (Shannon-Entropy). First, the individual step loss, after the application of a single depth-L layer stack, is given classically as Cross-Entropy Loss $\mathcal{L}^{(t)}$, or the negative log-likelihood of the ground-truth next token of the sequence x_{l+1} :

$$\mathcal{L}^{(t)} = \mathbb{E}_{x_{1:M}} \left[\sum_{\ell=1}^{M-1} -\log p_\theta^{(t)}(x_{\ell+1} | x_{1:\ell}) \right] \quad (4)$$

Given the single-step loss and the exit-step distribution $p_\theta(t | x)$ from Equation (3), our training objective combines next-token prediction with entropy regularization. After calculating the loss below, we update our learned parameters using a gradient descent update:

$$\mathcal{L}^{(t)} = \mathbb{E}_{x_{1:M}} \left[\sum_{\ell=1}^{M-1} -\log p_\theta^{(t)}(x_{\ell+1} | x_{1:\ell}) \right] \quad (5)$$

$$\mathcal{L} = \underbrace{\sum_{t=1}^{T_{\max}} p_\theta(t | x) \mathcal{L}^{(t)}}_{\text{expected task loss}} + \beta \underbrace{\sum_{t=1}^{T_{\max}} p_\theta(t | x) \log p_\theta(t | x)}_{\text{expected exit log-likelihood}} \quad (6)$$

4 Analysis & Open Questions

In summary, the objective or loss function defined in Equation (6) focuses the model on reducing the step loss of more probable steps, i.e. at more probable steps, we should ensure that the output of the LM head at the step maximizes the likelihood of the actual next token in the sequence. The authors use the cumulative density function of the learned step probabilities to determine when to exit:

$$\text{CDF}(n \mid x) = \sum_{t=1}^n p_{\phi}(t \mid x) = 1 - \prod_{j=1}^n (1 - \lambda_j(x)), \quad n < T_{\max}, \quad \text{CDF}(T_{\max} \mid x) = 1. \quad (7)$$

Given a threshold $q \in [0, 1]$, end recursion at first step where the cumulative probability crosses q :

$$t_{\text{exit}}(x) = \min \{m \in \{1, \dots, T_{\max}\} : \text{CDF}(m \mid x) \geq q\}. \quad (8)$$

The entropy regularization term in Equation (6) can be interpreted as either the expected log-likelihood of the exit distribution or as an entropy penalty that discourages collapse to deterministic exit decisions by penalizing low-entropy distributions. The authors further extend training in a **second stage** where exit gates learn to minimize the probability of exiting if the loss will improve with continued computation. Empirically, without entropy regularization, exit gates collapsed to either very early exits or always exiting at $t = T_{\max}$, likely because the probability-weighted formulation $\sum_t p_{\theta}(t \mid x) \mathcal{L}^{(t)}$ substantially down-weights later step losses as $p_{\theta}(t \mid x)$ shrinks. While entropy regularization addresses this bias, alternative loss formulations could achieve similar goals more directly, motivating the following open questions:

1. **Markovian exit decisions.** The current exit probability conditions on the full history through the survival function, creating dependencies on all previous steps. Could we impose a first-order Markov assumption where exit probability depends only on the previous step’s state and current hidden representation to simplify computation while maintaining adaptive depth?

$$p(t \mid x) \approx f(\lambda_{t-1}(x), h^{(t)}) \quad \text{instead of} \quad p(t \mid x) = \lambda_t(x) \prod_{j=1}^{t-1} (1 - \lambda_j(x))$$

2. **Depth-normalized loss alternative.** Since deeper computation typically improves loss, could minimizing the unweighted average loss naturally incentivize more steps, contrasting with the probability-weighted objective that may bias toward shallower exits? Could such depth normalization eliminate the need for two-stage training?

$$\mathcal{L}_{\text{avg}} = \frac{1}{T_{\max}} \sum_{t=1}^{T_{\max}} \mathcal{L}^{(t)} \quad \text{instead of} \quad \mathcal{L} = \sum_{t=1}^{T_{\max}} p_{\theta}(t \mid x) \mathcal{L}^{(t)}$$

3. **Explicit depth penalty.** While entropy regularization indirectly controls computational cost, could we add an explicit depth penalty to directly discourage excessive computation and improve generalization by preventing over-reliance on depth?

$$\mathcal{L}_{\text{depth}} = \mathcal{L} + \gamma \sum_{t=1}^{T_{\max}} t \cdot p_{\theta}(t \mid x)$$

5 Aside: Mixture of Experts Routing Algorithms

Mixture of Experts (MoE) is now readily adopted as the standard position-wise feedforward network of the modern LLM architecture, conventionally incorporated with the pre-norm, MHSA+RoPE, and residual streams. The challenge of training MoEs is how to learn which sparse network should be used to transform a given input embedding of shape: (1, 1, model-dim). During training a model must learn to dynamically route the embedding to the right transformation, and to do so sparsely. How is this done in practice? The primary approach (Mixtral, DBRX, DeepSeek v3) is called **top-k routing**.

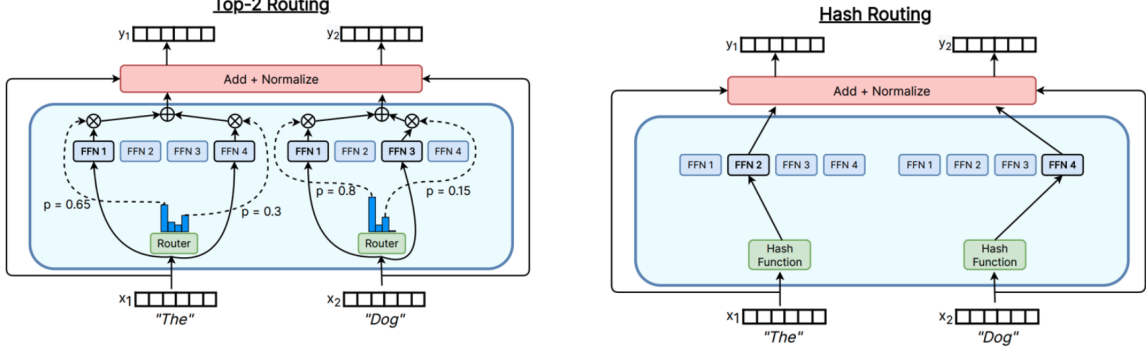


Figure 4: Left: Top-K Routing. Right: Hash Routing

The equations below are descriptions of top-k routing. In summary, the i 'th feed forward network is applied to the input embedding $\mathbf{u}_t^l$ if it is one of the top-k most similar. The i 'th feedforward network is represented by a learned representation $\mathbf{e}_i^l$. Notice the application of softmax on the dot product of $\mathbf{u}_t^l$ and $\mathbf{e}_i^l$: this just means that we form a probability distribution over all experts for a given input embedding $\mathbf{u}_t^{lT}$.

$$\mathbf{h}_t^l = \sum_{i=1}^N (g_{i,t} \text{FFN}_i(\mathbf{u}_t^l)) + \mathbf{u}_t^l, \quad (9)$$

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise,} \end{cases} \quad (10)$$

$$s_{i,t} = \text{Softmax}_i(\mathbf{u}_t^{lT} \mathbf{e}_i^l), \quad (11)$$

6 Aside: MoE Load Balancing and Z losses

When training sparse activation via routers, there are two sometimes competing objectives: making sure that the embedding goes to the best router, and making sure that all the individual feed-forward networks get trained. If the transformer collapses to using only one expert, that expert (often stored on a single GPU) can get assigned too many embeddings in the batch and this can bog down inference. The following two auxiliary losses are used together in practice: a **load balancing loss**, and a **Z-loss**.

$$\mathcal{L}_{LB} = N_E \cdot \sum_{i=1}^{N_E} f_i \cdot P_i \quad (12)$$

where $\mathcal{L}_{LB}$ is the load balancing loss, N_E is the number of experts, f_i is the fraction of tokens routed to expert E_i , and P_i is the routing probability allocated to E_i .

$$\mathcal{L}_{RZ}(x) = \frac{1}{B} \cdot \sum_{i=1}^B \left(\log \sum_{j=1}^{N_E} \exp(x_j^{(i)}) \right)^2 \quad (13)$$

where B is the batch size, $x_j^{(i)}$ is the logit for expert j for sample i , and β is an optional loss weight (commonly 0.001). This penalizes any one expert from being weighted too highly for embeddings.

7 Analogy: MoE in the Layer Space

Instead of learning a fixed stack of layers and repeating it for N iterations, to improve expressivity, we can imagine learning a handful of stacks, and dynamically routing (but re-using) the right stack given an input embedding. This is what I'm interested in developing.

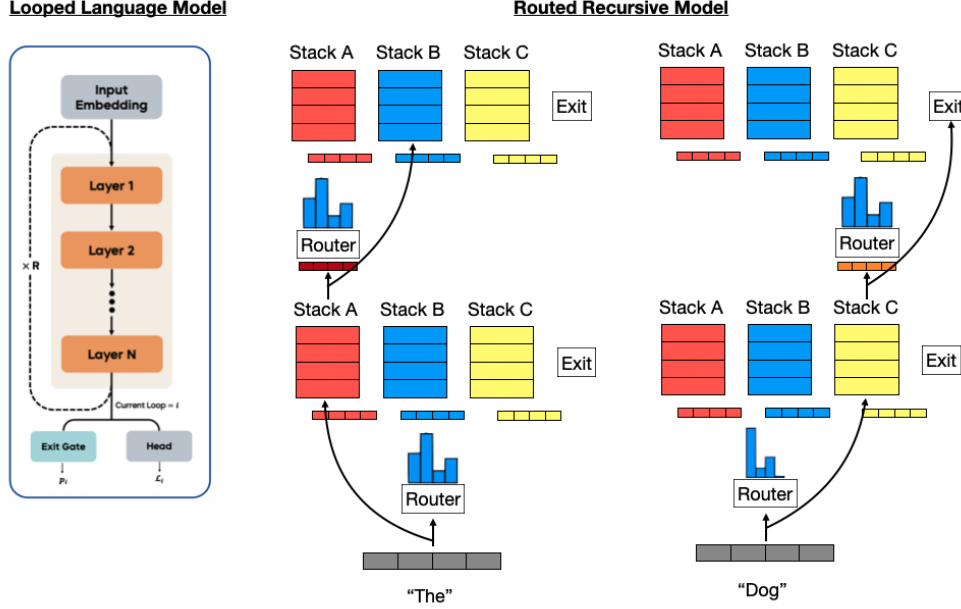


Figure 5: Routed Recursive Models

8 A Small Test of Recursive Models

To test the expressiveness of recursive models, I compared a 6-layer transformer (28.9M params) with a recursive model repeating a 2-layer stack 3 times (16M params).¹ Both were pretrained on TinyStories for 32M tokens with identical model dimensions and optimizer settings. The recursive model matched the full model's convergence despite being 1.8x smaller, demonstrating comparable expressiveness.

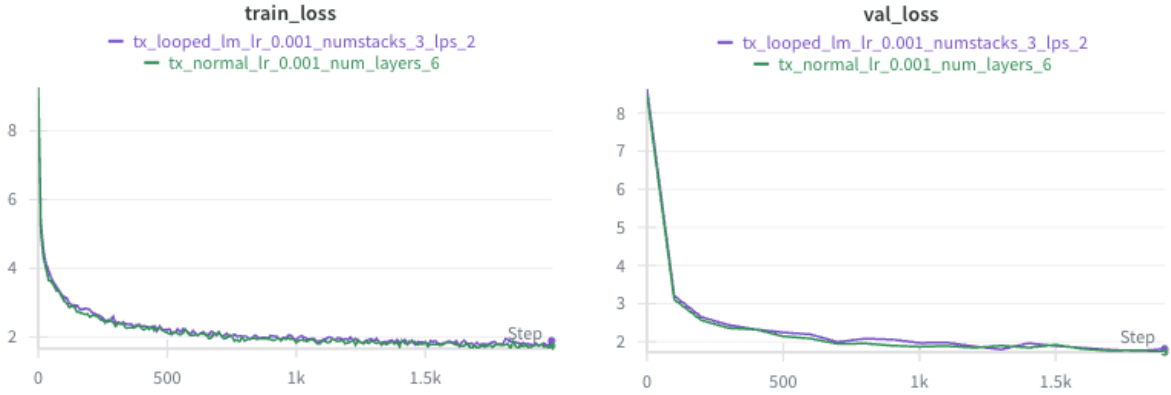


Figure 6: Training loss (left) and validation loss (right) compared for a standard transformer vs. a recursive model.

¹The baseline transformer uses a modern architecture: residual stream, RMSNorm, multi-head self-attention, and SwiGLU activation.

Appendix I Recurrent-Depth Model Results

Table 4. % Test accuracy on Puzzle Benchmarks (Sudoku-Extreme and Maze-Hard)

Method	# Params	Sudoku	Maze
Chain-of-thought, pretrained			
Deepseek R1	671B	0.0	0.0
Claude 3.7 8K	?	0.0	0.0
O3-mini-high	?	0.0	0.0
Direct prediction, small-sample training			
Direct pred	27M	0.0	0.0
HRM	27M	55.0	74.5
TRM-Att (Ours)	7M	74.7	85.3
TRM-MLP (Ours)	5M/19M ¹	87.4	0.0

Table 5. % Test accuracy on ARC-AGI Benchmarks (2 tries)

Method	# Params	ARC-1	ARC-2
Chain-of-thought, pretrained			
Deepseek R1	671B	15.8	1.3
Claude 3.7 16K	?	28.6	0.7
o3-mini-high	?	34.5	3.0
Gemini 2.5 Pro 32K	?	37.0	4.9
Grok-4-thinking	1.7T	66.7	16.0
Bespoke (Grok-4)	1.7T	79.6	29.4
Direct prediction, small-sample training			
Direct pred	27M	21.0	0.0
HRM	27M	40.3	5.0
TRM-Att (Ours)	7M	44.6	7.8
TRM-MLP (Ours)	19M	29.6	2.4

Figure 7: Tiny Recursive Model performance on puzzle tasks (left) and ARC-AGI benchmarks (right)

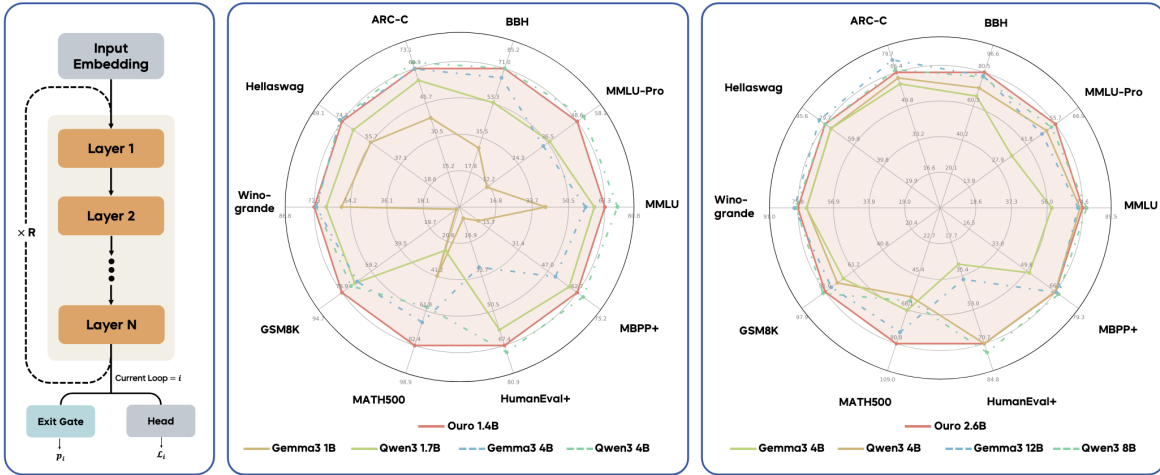


Figure 8: LoopedLM (Ouro) performance on standard benchmarks compared to much larger models.

References

- [JM25] Alexia Jolicoeur-Martineau. Less is More: Recursive Reasoning with Tiny Networks, October 2025. arXiv:2510.04871 [cs].
- [WLS⁺25] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical Reasoning Model, August 2025. arXiv:2506.21734 [cs].
- [ZWH⁺25] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling Latent Reasoning via Looped Language Models, November 2025. arXiv:2510.25741 [cs].